Comp 512
Spring 2012

Cleaning up the CFG

Eliminating Useless Nodes and Edges

This lecture describes the algorithm Clean, presented in Chapter 10 of EaC2e. The algorithm is due to Rob Shillingsburg, when he worked on the MSCP at Rice.

Copyright 2012, Keith D. Cooper & Linda Torczon, all rights reserved.

Students enrolled in Comp 512 at Rice University have explicit permission to make copies of these materials for their personal use.

Faculty from other educational institutions may use these materials for nonprofit educational purposes, provided this copyright notice is preserved.

Dead Code Elimination

Three distinct problems

- Useless operations
  - → Any operation whose value is not used in some visible way
  - → Use the SSA-based mark/sweep algorithm (DEAD)
- Useless control flow
  - → Branches to branches, empty blocks
  - → Simple CFG-based algorithm (Shillingsburg's CLEAN)
- Unreachable blocks
  - → No path from n_0 to $b \Rightarrow b$ cannot execute
  - → Simple graph reachability problem

Eliminating Useless Control Flow

The Problem

- After optimization, the CFG can contain empty blocks
- "Empty" blocks still end with either a branch or a jump
- Produces jump to jump, which wastes time & space
- Need to simplify the CFG & eliminate these

We must distinguish between these
- Branch is conditional
- Jump is absolute

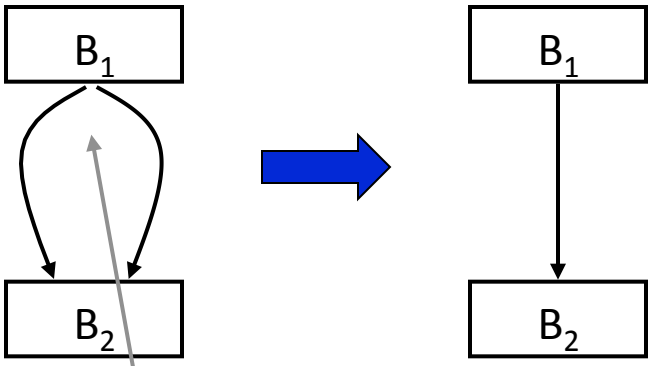
The Algorithm (CLEAN)

- Use four distinct transformations
- Apply them in a carefully selected order
- Iterate until done

Devised by Rob Shillingsburg (1992), documented by John Lu (1994)

Eliminating Useless Control Flow

Transformations



Eliminating redundant branches

Branch, not a jump

Both sides of branch target B_i

- Neither block must be empty
- Replace it with a jump to B_i
- Simple rewrite of last op in B_1

How does this happen?

- Rewriting other branches

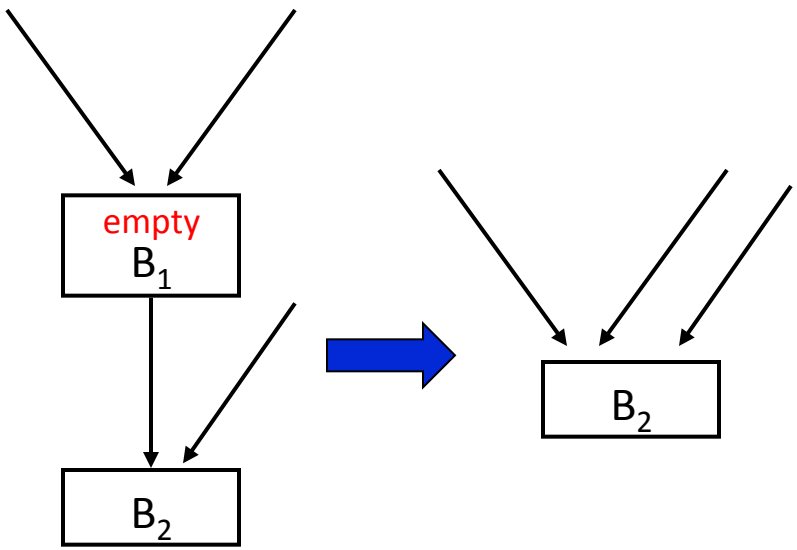
How do we find it?

- Check each branch

*

Eliminating Useless Control Flow

Transformations



Eliminating empty blocks

Merging an empty block

- Empty B_1 ends in a jump
- Coalesce B_1 with B_2
- Move B_1's incoming edges
- Eliminates extraneous jump
- Faster, smaller code

How does this happen?

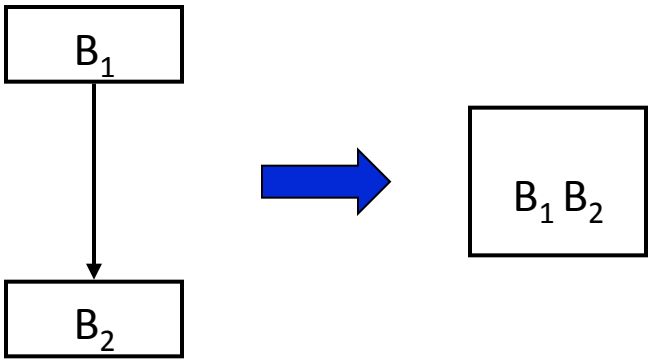
- Eliminate operations in B_1

How do we find it?

- Test for empty block

Eliminating Useless Control Flow

Transformations



Combining non-empty blocks

B_1 and B_2 should be a single basic block

If one executes, both execute, in linear order.

Coalescing blocks

- Neither block must be empty
- B_1 ends with a jump
- B_2 has 1 predecessor
- Combine the two blocks
- Eliminates a jump

How does this happen?

- Simplifying edges out of B_1

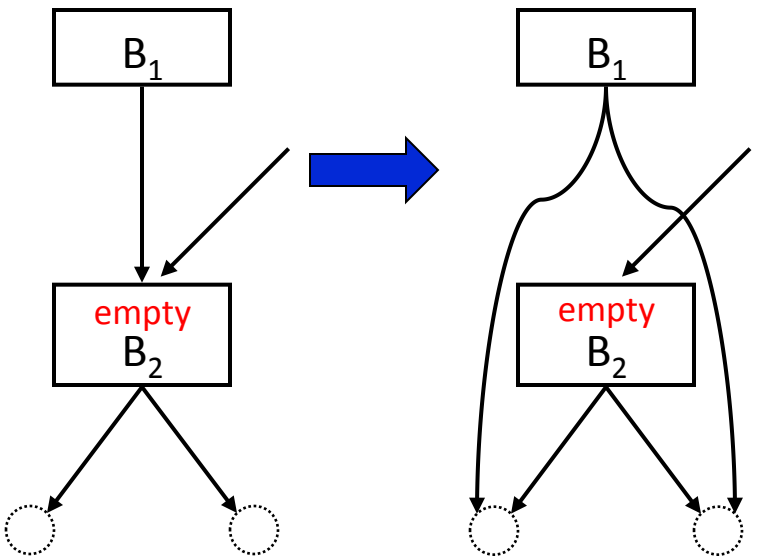
How do we find it?

- Check target of jump *|preds |*

*

Eliminating Useless Control Flow

Transformations



Hoisting branches from empty blocks

Jump to a branch

- B_1 ends with jump, B_2 is empty
- Eliminates pointless jump
- Copy branch into end of B_1
- Might make B_2 unreachable

How does this happen?

- Eliminating operations in B_2

How do we find this?

- Jump to empty block

Eliminating Useless Control Flow

Putting the transformations together

- Process the blocks in postorder
  - → Clean up B_i's successors before B_i
  - → Simplifies implementation & understanding
- At each node, apply transformations in a fixed order
  - → Eliminate <u>redundant</u> branch ← Eliminates an edge
  - → Eliminate <u>empty</u> block ← Elim
  - → Merge block with successor ← Elim
  - → Hoist branch from empty successor ← Add

Handles jump

Creates a jump that should be checked in the same pass

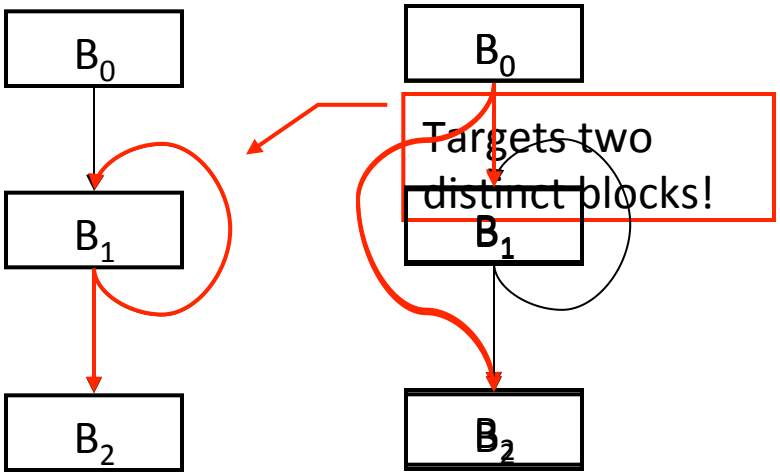
- May need to iterate
  - → Postorder ⇒ unprocessed successors along back edges
  - → Can bound iterations, but a deriving tight bound is hard
  - → Must recompute postorder between iterations

*

Eliminating Useless Control Flow

What about an empty loop?

- By itself, CLEAN cannot eliminate the loop
- Loop body branches to itself
  - → Branch is <u>not</u> redundant *
  - → Doesn't end with a jump
  - → Hoisting does not help *
- Key is to eliminate self-loop
  - → Add a new transformation? *
  - → Then, B_1 merges with B_2 *



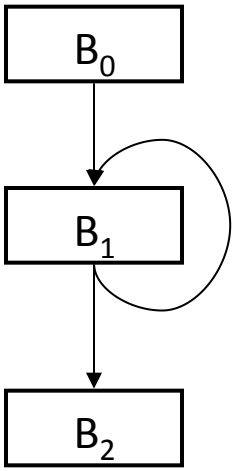
New transformation must recognize that B_1 is empty. Presumably, it has code to test exit condition & (probably) increment an induction variable.

This requires looking at code inside B_1 and doing some sophisticated pattern matching. This seems awfully complicated.

Eliminating Useless Control Flow

What about an empty loop?

- How to eliminate <B_1,B_1> ?
  - → Pattern matching ?
  - → Useless code elimination ?

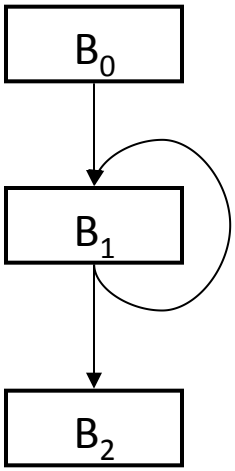


*

Eliminating Useless Control Flow

What about an empty loop?

- How to eliminate $<B_1,B_1>$?
  - → Pattern matching ?
  - → Useless code elimination ?
- What does DEAD do to B_1?
  - → Remember, it is empty
  - → Contains only the branch
  - → B_1 has only one exit
  - → So, $B_1 \notin RDF(B_2)$
  - → B_1's branch is <u>useless</u>
  - → DEAD rewrites it as a jump to B2



*

Using SSA – Dead code elimination

Shillingsburg added this clause.

Mark

```
for each op i
    clear i's mark
    if i is critical then
        mark i
        add i to WorkList

while (Worklist ≠ Ø)
    remove i from WorkList
        (i has form "x←y op z")
    if def(y) is not marked then
        mark def(y)
        add def(y) to WorkList
    if def(z) is not marked then
        mark def(z)
        add def(z) to WorkList

    for each b ∈ RDF(block(i))
        mark the block-ending
            branch in b
        add it to WorkList
```

Sweep

```
for each op i
    if i is not marked then
        if i is a branch then
            rewrite with a jump to i's
            nearest useful post-dominator
        if i is not a jump then
            delete i
```

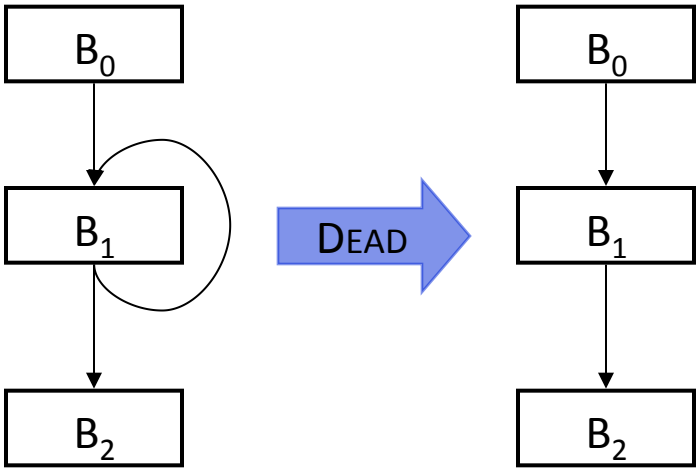
Notes:

- Eliminates some branches
- Reconnects dead branches to the remaining live code
- Find useful post-dominator by walking post-dom tree
  - Entry & exit nodes are always "useful"

Eliminating Useless Control Flow

What about an empty loop?

- How to eliminate $<B_1,B_1>$?
  - → Pattern matching ?
  - → Useless code elimination ?
- What does DEAD do to B_1?
  - → Remember, it is empty
  - → Contains only the branch
  - → B_1 has only one exit
  - → So, $B_1 \notin RDF(B_2)$
  - → B_1's branch is <u>useless</u>
  - → DEAD rewrites it as a jump to B2



DEAD converts it to a form where CLEAN handles it !

Eliminating Useless Control Flow

The Algorithm

```
CleanPass()
    for each block i, in postorder
        if i ends in a branch then
            if both targets are identical then
                rewrite with a jump
        if i ends in a jump to j then
            if i  is empty then
                merge i with j
            else if j has only one predecessor
                merge i with j
            else if j is empty & j has a branch then
                rewrite i's jump with j's branch

Clean()
    until CFG stops changing
        compute postorder
        CleanPass()
```

Summary

- Simple, structural algorithm
- Limited transformation set
- Cooperates with DEAD
- In practice, its quite fast

Eliminating Unreachable Code

The Problem

- Block with no entering edge
- Situation created by other optimizations

The Cure

- Compute reachability & delete unreachable code
- Simple mark/sweep algorithm on CFG
- Mark during computation of postorder, reverse postorder ...
- In MSCP, importing ILOC does this (every time)

Dead Code Elimination

Summary

- Useless Computations ⇒ DEAD
- Useless Control-flow ⇒ CLEAN
- Unreachable Blocks ⇒ Simple housekeeping

Other Techniques

- Constant propagation can eliminate branches
- Algebraic identities eliminate some operations
- Redundancy elimination
  - → Creates useless operations, or
  - → Eliminates them